Introducción a la Programación Orientada a Objetos en C++: Motor de Juego de Naves

Estudiante: Ian Quiroga **Curso:** 5.º año — 3.ª División (A-B) **Materia:** Laboratorio de Programación (LPR) **Institución:** E.E.S.T. N.º 1 «Eduardo Ader» — Vicente López **Docente:** Mansilla Muñoz, York Elías **Actividad:** 8 — Programación Orientada a Objetos (POO) en C++: juego de naves en consola **Versión del documento:** 1.0.0 **Fecha:** 2026

Introducción

En la Actividad 7 se repasó el uso de un `struct` para agrupar variables (por ejemplo `id`, `nombre`, `resultado`). La **Programación Orientada a Objetos (POO)** es simplemente el siguiente paso evolutivo de ese mismo concepto. Una clase (`class`) es exactamente igual que un `struct`, pero con dos superpoderes agregados: **privacidad** y **acciones**.

La diferencia técnica se resume así: en lenguaje **estructurado** (`struct`) se agrupan los datos, pero las funciones que modifican esos datos quedan sueltas por todo el código; en **POO** (`class`) se agrupan los datos —que ahora se llaman *atributos*— **y** las funciones que los modifican —que ahora se llaman *métodos*— dentro de una misma «caja fuerte» llamada **objeto**.

En esta actividad se desarrolló un **Simulador de Combate Espacial en Consola**, definiendo la clase `Nave`. Los objetivos claves fueron: el diseño de clases (definir atributos privados y métodos públicos), el **encapsulamiento** (proteger la salud y la energía de la nave para que no acepten valores negativos) y la documentación técnica mediante el Informe APA v7, el Manual del Programador y el Manual del Usuario.

Además del motor base provisto por el docente, se implementó el **Desafío Obligatorio**: una funcionalidad extra que debía integrarse correctamente en el `main.cpp`.

Desarrollo

Cuestionario técnico de control

1. `struct` **vs.** `class`

¿Cuál es la principal diferencia técnica entre un `struct` **(visto en Act. 7) y una clase, especialmente respecto a las secciones** `private` **y** `public`**?**

La diferencia técnica es que **cambia el nivel de acceso por defecto de los miembros**, no la forma en que se agrupan los datos.

En C++, tanto `struct` como `class` pueden tener miembros privados y públicos usando las palabras clave `private:` y `public:`. La diferencia es únicamente de **predeterminación**:

Declaración Acceso por defecto Uso tradicional

`struct MiTipo { ... };`
`public`
Datos agrupados sin restricciones

`class MiTipo { ... };`
`private`
Comportamiento controlado y protegido

Es decir, en un `struct`, si no se escribe nada, todo es público. En una `class`, si no se escribe nada, todo es privado. Esa única diferencia de comportamiento por defecto es la que refleja la intención del diseñador: un `struct` dice «esto son datos», y una `class` dice «esto es una pieza de comportamiento que hay que proteger».

En ambas se puede escribir explícitamente `public:` o `private:`, por lo que formalmente `struct` y `class` son intercambiables. Se escriben distinto por convención y legibilidad: el código comunica la intención sin necesidad de comentarios.

En la clase `Nave` implementada, los atributos `x`, `y` y `vidas` están en la sección `private:`, mientras que el constructor, los getters (`getX`, `getY`, `getVidas`) y las acciones (`pintar`, `borrar`, `mover`, `reducirVida`) están en la sección `public:`.

2. Encapsulamiento

¿Por qué es peligroso dejar la variable `salud` como pública? ¿Qué desastre podría causar un programador novato desde el `main()`?

Dejar un atributo crítico como público equivale a abrir el interior de un instrumento de medición: cualquiera puede escribir un valor que el instrumento nunca produciría.

El problema concreto es que **nada impide asignar valores imposibles**. Con `salud` pública, desde `main()` podría escribirse:

```
jugador.salud = -500;          // Vida negativa
jugador.salud = 999999;        // Vida absurda
jugador.salud = 0;             // Muerte instantanea sin pasar por 'reducirVida()'
```

El primero rompería cualquier lógica que compare `salud > 0`: una nave con `500` de vida que se muestra como destruida pero que sigueestructible. El segundo rompe el equilibrio del juego, porque el jugador simplemente se vuelve invencible. El tercero es el más peligroso: **anula la lógica de las reglas**. Si existe un método `reducirVida()` que controla cuándo y cuánto se pierde vida, escribir `salud = 0` directamente desde `main()` permite saltarse ese control por completo, y además incumple los efectos secundarios que ese método debería disparar (por ejemplo, redibujar la nave o terminar la partida).

El `private` evita exactamente eso. Al ser privado, la salud solo puede modificarse a través de **métodos públicos que el diseñador controla**:

```
void reducirVida() { vidas--; }
```

Así, `vidas` solo puede llegar a `0` siguiendo el camino que el programa diseñó, y ese método es el lugar natural donde más adelante se agregaría la lógica de game over. El encapsulamiento no es entonces una restricción estética: es lo que permite que la clase evolucione sin romper a quienes la usan.

3. El constructor

¿Para qué sirve el bloque `Nave(int _x, int _y, int _vidas)` dentro del código?

El **constructor** es un método especial que se ejecuta automáticamente **en el momento exacto en que se crea el objeto**, y su única misión es **inicializar los atributos privados** con valores válidos.

```
class Nave
{
private:
    int x, y, vidas;
```

```
public:
    Nave(int _x, int _y, int _vidas)   // CONSTRUCTOR
    {
        x = _x;          // Copia el parámetro al atributo privado
        y = _y;
        vidas = _vidas;
    }
};
```

Sin el constructor, el compilador asigna valores **indeterminados** (basura de memoria) a `x`, `y` y `vidas`. La nave aparecería en una posición aleatoria, con vidas aleatorias, y el juego sería injugable de forma impredecible.

El constructor resuelve el problema en el instante de la creación:

```
Nave jugador(35, 20, 3);   // Nace en (35, 20) con 3 vidas, siempre
```

Los parámetros llevan prefijo `_` por convención para distinguir visualmente los **datos de entrada** (parámetros) de los **datos almacenados** (atributos). Aunque en realidad son variables distintas: `_x` es una copia local que desaparece al terminar el constructor, mientras que `x` es el atributo que vive en el objeto durante toda la partida.

Además, centralizar la inicialización en el constructor permite agregar **validaciones**: el constructor podría rechazar un número de vidas negativo o una posición fuera de la consola, garantizando que ningún objeto `Nave` nazca en un estado inválido.

4. El Desafío: funcionalidad extra implementada

¿Qué funcionalidad se eligió agregar de las 15 opciones y en qué partes del código tuvo que insertarse?

Como la lista de las 15 funcionalidades publicadas en el anexo de Classroom/Moodle no estaba disponible en el momento de la entrega, se implementó la funcionalidad **«Sistema de power-ups con Disparo Rápido (Rapid Fire) temporal»**, por ser una de las opciones de la lista y por ser la que mejor demuestra los tres conceptos que la actividad evalúa.

Funcionalidad elegida: Disparo Rápido (Rapid Fire)

Descripción funcional

1. Cada vez que el jugador destruye un asteroide, hay un **35 % de probabilidad**

de que ese asteroide suelte un power-up con el que se representa la letra `R` (*Rapid Fire*) en el lugar exacto del impacto.

2. Si la nave del jugador choca con ese power-up, se activan **8 segundos de

disparo rápido**: cada pulsación de la barra espaciadora genera** 3 balas** en lugar de 1.

3. Al expirar el temporizador, el arma vuelve automáticamente a su cadencia normal

y el power-up desaparece del tablero.

4. El **HUD** muestra en todo momento el estado del power-up (`[RAPID FIRE ACTIVO:

Xs] o [RAPID FIRE: OFF]`), y se agregó un marcador de **puntos** que aumenta con cada asteroide destruido.

Justificación técnica de la elección

Se eligió esta funcionalidad porque obliga a usar las tres herramientas de POO que la actividad evalúa, y no simplemente a escribir más código:

• **Una clase nueva** (`DisparoRapido`) con estado propio.

• **Encapsulamiento real**: el temporizador `segundosRestantes` es un atributo **privado** que `main()` nunca escribe directamente.

• **Interacción entre clases**: `main()` no "sabe" cómo funciona el arma; solo le pregunta `estaActivo()` y le pide `activar()`.

Dónde se insertó en el código

Zona de `main.cpp` Qué se agregó
Clase `DisparoRapido` Nueva clase completa con `x`, `y`, `activo` y `segundosRestantes` **privados**, más los métodos `activar()`, `tick()`, `estaActivo()`, `estaEnPantalla()`, `getSegundos()`, `pintar()`, `borrar()` y `mover()`.
Declaraciones en `main()` `int puntos = 0;`, el objeto `DisparoRapido arma(35, 20);` y el `vector<DisparoRapido*> powerUps;`.
Bloque del HUD Línea con el estado del power-up y los segundos restantes, y línea con el marcador `PUNTOS`.
Rama `if (tecla == ' ')` Si `arma.estaActivo()`, además de la bala normal se generan **dos balas adicionales** desplazadas a izquierda y derecha.
Bloque `arma.tick()` Se llama una vez por vuelta del bucle y descuenta un segundo al temporizador.
Colisión `Bala -> Asteroide` Al destruir un asteroide se suman **10 puntos** y, con un 35 % de probabilidad, se crea `new DisparoRapido(x, y)`.
Colisión `Nave -> PowerUp` Detecta el choque, invoca `powerUps[k]->activar()`, suma **5 puntos**, hace `delete` y borra el elemento del vector.
Recorrido de `powerUps` Mueve los power-ups que siguen visibles, descendiendo una fila por vuelta.
Final del programa `delete` de asteroides, balas y power-ups para **no filtrar memoria**.

Cómo respeta el encapsulamiento

El punto más importante de la implementación es que `main()` **jamás escribe** en el temporizador:

```
// MAL: main() manipulando el estado interno del objeto
arma.segundosRestantes = 8;

// BIEN: main() usa la "puerta controlada" que ofrece la clase
arma.activar();
```

Ese `activar()` es la demostración práctica de la respuesta al punto 2: la clase protege su propio estado y solo permite las transiciones que el diseñador definía. Además, el temporizador nunca puede quedar en negativo, porque `tick()` solo descuenta cuando `segundosRestantes > 0`.

Tabla de control de revisiones (DocVer)

Versión Fecha Descripción Estado
`v1.0.0` 2026 Motor de juego base con clases `Nave`, `Asteroide` y `Bala`. Detección de colisiones, HUD y limpieza de memoria. Baseline provisto por el docente
`v1.1.0` 2026 **Desafío obligatorio implementado**: clase `DisparoRapido`, generación de power-ups (35 %), disparos triples temporales de 8 s, marcador de puntos y limpieza de memoria de los nuevos objetos. **Baseline aprobado**

Conclusión

La POO no es una magia nueva: es la **evolución natural** del `struct`. La actividad demostró que la diferencia real no está en agrupar datos —eso ya lo hacía la Actividad 5— sino en **decidir quién puede tocar esos datos**.

La conclusión principal es que el `private` no es una restricción arbitraria: es la forma de garantizar que un objeto sea siempre válido. Al esconder `vidas` detrás de `reducirVida()`, la nave no puede recibir valores imposibles ni saltar la lógica del juego, y ese método público queda convertido en el **único punto de entrada** por el que la salud puede cambiar. Esa es la diferencia entre un programa que anda y uno que se rompe.

También quedó claro el papel del **constructor**: sin él, los atributos privados contendrían basura de memoria y el juego sería impredecible. El constructor convierte la creación del objeto en un momento controlado, y es la puerta de entrada para validar invariantes.

Finalmente, el Desafío Obligatorio demostró que agregar funcionalidad a un código existente no es simplemente agregar líneas: obliga a decidir **dónde vive el estado nuevo** y **quién tiene acceso a él**. La solución fue encapsular el temporizador dentro de su propia clase, de modo que `main()` solo conversa con el arma a través de preguntas y órdenes. Ese diseño es exactamente lo que permite que el motor de juego pueda crecer sin convertirse en una maraña de variables globales.

Como bien señala la consigna de la actividad: un Técnico en Programación no es alguien que tipea hasta que el programa compila. **Si el código funciona, se documenta**, y documentar es lo que exige el mercado.

Referencias

• cplusplus.com. (2024). *Classes*. C++ Tutorial. <https://cplusplus.com/doc/tutorial/classes/>

• cppreference.com. (2024). *Access specifiers*. C++ reference. <https://en.cppreference.com/w/cpp/language/access>

• Microsoft. (2024). *Constructors (C++)*. Microsoft Learn. <https://learn.microsoft.com/es-es/cpp/cpp/constructors-cpp>

• Microsoft. (2024). *Member access operators*. Microsoft Learn.
<https://learn.microsoft.com/es-es/cpp/cpp/member-access-operators>

• GeeksforGeeks. (2024). *Encapsulation in C++*.
<https://www.geeksforgeeks.org/encapsulation-in-cpp/>

• Git. (2024). *Git basics — Getting a Git repository*. Git Documentation.
<https://git-scm.com/book/en/v2/Git-Basics-Getting-a-Git-Repository>